

# Persiapan Lab 3: Naive Algorithm

IBDA2011 - dipersiapkan oleh: Yozef Tjandra

## Tujuan

Tiket masuk ini bertujuan untuk mempersiapkan Anda menerapkan strategi Algoritma Naif dalam penyelesaian masalah.

## Instruksi

Buatlah program untuk menyelesaikan masalah yang sudah disediakan. Program tidak perlu dikumpulkan, tapi jika Anda mengerjakannya dengan tuntas, Anda akan bisa 'curi start' untuk mengerjakan tugas-tugas dalam sesi Lab dengan lebih lancar.

## Masalah 1: Pasangan Titik Terdekat

Tuliskan program untuk menyelesaikan masalah Pasangan Titik Terdekat dalam Bidang Cartesius dengan strategi Exhaustive Search.

Ingat kembali bahwa dalam bidang Cartesius, jarak antar dua buah titik  $P_1 = (x_1, y_1)$  dan  $P_2 = (x_2, y_2)$  didefinisikan oleh jarak Euclid sebagai berikut:

$$d(P_1, P_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Dalam masalah ini, Anda akan diberikan input berupa dua buah vektor  $X$  dan  $Y$  yang masing-masing berukuran  $n$  dan berisi bilangan real. Dua buah vektor tersebut menyatakan  $n$  buah titik pada bidang Cartesius, yaitu  $(x[0], y[0]), (x[1], y[1]), \dots, (x[n-1], y[n-1])$ . Anda diminta untuk menghasilkan output berupa **semua** pasangan titik yang memiliki jarak terdekat dari antara semua pasangan titik lainnya.

Input : Two lists  $x$  and  $y$  of  $n$  ( $n \geq 2$ ) points  $p_1(x_1, y_1), \dots, p_n(x_n, y_n)$   
Output : The distance between the closest pair of points and all pairs of points attaining that distance.

Berikut merupakan contoh Input dan Output permasalahan.

Input:

```
2.0 -3.0 5.5 6.5 1.0 0.5 0
0.0 5.5 1.5 -0.5 0.5 3.5 4.5
```

Output:

```
1.118
0 4
5 6
```

Pada contoh di atas, jarak terpendek di antara semua pasangan titik adalah sebesar 1.118, yang dicapai oleh 2 buah pasangan titik:

1. Pasangan antara titik ke-0, yaitu (2,0) dan titik ke-4, yaitu (1, 0.5); dan
2. Pasangan antara titik ke-5, yaitu (0.5, 3.5) dan titik ke-6, yaitu (0, 4.5).

## Masalah 2: Perkalian Matriks

Masalah perkalian matriks merupakan salah satu masalah numerik yang paling klasik dan merupakan suatu latihan yang sangat baik untuk mengembangkan kemampuan berpikir abstrak seorang pemrogram pemula. Berikut merupakan deskripsi masalahnya.

**Input** : Matrix A of size  $m \times n$  and Matrix B of size  $n \times t$

**Output** : Matrix AB of size  $m \times t$ , the result of multiplying A and B

Misalkan  $A \in \mathbb{R}^{m \times n}$  dan  $B \in \mathbb{R}^{n \times t}$ . Misalkan pula  $C = AB$ , hasil perkalian kedua matriks. Maka  $C \in \mathbb{R}^{m \times t}$ .

Untuk dapat menghitung matriks  $C$ , maka tentu diperlukan perulangan sebanyak  $m \times t$  kali untuk menghitung setiap elemen dari matriks  $C$ . Setiap elemen hasil perkalian matriks tersebut didefinisikan melalui formula berikut:

$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad \text{untuk semua } i = 1, 2, \dots, m; \quad j = 1, 2, \dots, t.$$

Maka, perhitungan perkalian matriks yang dilakukan secara naif akan menerapkan secara langsung formula di atas untuk setiap elemen matriks  $C$ .

## Masalah 3: Brute Force String Matching

Permasalahan *String Matching* merupakan salah satu masalah paling klasik dalam ilmu komputer. Secara umum, masalah ini mirip dengan masalah *Searching*, namun dengan tingkat kompleksitas yang lebih. Masalah ini menerima dua buah string **text** dan **pattern** sebagai input untuk mencari tahu apakah string **pattern** muncul sebagai sub-string dari **text**. Perhatikan contoh berikut.

```
text    = "malam minggu merajam maling"
pattern = "raja"
```

**Output** = TRUE

Output di atas adalah TRUE karena `pattern = text[15:4]`.

```
text    = "malam minggu merajam maling"
pattern = "malas"
```

**Output** = FALSE

Algoritma naif untuk menyelesaikan permasalahan ini sangat mirip dengan *Sequential Search*, yaitu mengecek satu demi satu setiap posisi string **text** kemudian menelusuri setiap digit string berikutnya apakah berisi sama dengan string **pattern**, digit demi digit.

Input: An array  $T$   $[0..n - 1]$  of  $n$  characters representing a text and an array  $P$   $[0..m - 1]$  of  $m$  characters representing a pattern  
 Output: The index of the first character in the text that starts a matching substring or  $-1$  if the search is unsuccessful

## Masalah 4: Brute Force Convex Hull

Permasalahan Convex Hull jika diselesaikan secara *Exhaustive Search* akan memiliki kerangka algoritma sebagai berikut.

Input : Two lists  $x$  and  $y$  of  $n$  ( $n \geq 2$ ) points  $p_1(x_1, y_1), \dots, p_n(x_n, y_n)$

Output : The total length of the convex hull surrounding the points on the input

```
ConvexHull(x, y):
    ConvexHullLength = 0
    for i = 1, 2, ... , n:
        for j = i, i+1, ... , n:
            if IncludedInConvexHull(x[i], y[i], x[j], y[j]):
                ConvexHullLength += Distance(x[i], y[i], x[j], y[j])
    return ConvexHullLength
```

1. Tulislah sebuah fungsi **CrossProdCheck()** yang menerima input berupa tiga buah titik  $A, B, C$  di bidang Cartesius yang akan mengembalikan nilai 1 jika vektor  $\vec{AB}$  berada di sebelah kanan vektor  $\vec{AC}$ ; mengembalikan nilai  $-1$  jika vektor  $\vec{AB}$  berada di sebelah kiri vektor  $\vec{AC}$ ; dan mengembalikan nilai 0 jika vektor  $\vec{AB}$  dan vektor  $\vec{AC}$  terletak pada satu garis yang sama.
2. Tulislah sebuah fungsi **ClassifyPoints()** yang menerima input berupa  $n$  buah titik di bidang Cartesius dan dua buah titik  $A$  dan  $B$ . Fungsi ini bertujuan untuk menyortir  $n$  buah titik inputan dalam hal posisi titik tersebut terhadap segmen garis yang dibentuk oleh titik  $A$  dan  $B$ . Fungsi mengembalikan sebuah *dictionary* yang menerjemahkan masing-masing titik menjadi tiga buah nilai: 1 jika titik di sebelah kanan ruas garis,  $-1$  jika titik di sebelah kiri ruas garis dan 0 jika terletak pada ruas garis/perpanjangannya.
3. Tulislah fungsi **IncludedInConvexHull()** pada algoritma utama **ConvexHull()** untuk mendeteksi apakah sebuah pasangan titik termasuk ke dalam convex hull atau tidak.
4. Tulislah program lengkap **ConvexHull()**.